IA explicable y reglas difusas: ¿Sinergia, redundancia, inutilidad o antagonismo?

Subtítulo

Autor: Jorge Montesinos Bravo
Tutor: Mercedes Valdés Vela

Fecha: 99/05/2026

Motivación: las reglas difusas han sido tradicionalmente modelos interpretables. Igual que la IA vivió su invierno, y ahora ha resurgido como el Avel Fénix, últimamente es el Soft computing y, concretamente la Lógica Fuzzy lo que han sido denostados por diferentes razones, entre otras que el deep learning lo ha fagocitado todo (gracias, a cantidades de deatos, potencia de cálculo, velocidad). Sin embarog tiene el problema de la explicabilidad: en este tfm se estudia la posiblidad de que las reglas difusas puedan encontrar su lugar en esta nueva elra de la IA aportando a la explicabilidad. Este viaje puede ser un fracaso por lo que el título del TFM es una pregunra: sinergia, redundancia o antagonismo?

CONTEXTUALIZACIÓN O ESTADO DEL ARTE

```
CONTEXTUALIZACIÓN FUZZY

      SISTEMAS DE REGLAS DIFUSAS Y SOFT COMPUTING

            (fuente: mi tesis)

            EL INVIERNO DEL SOFT COMPUTING

                básicamente el deep learning lo acapara todo

                    problema de estas técnicas: no explicabilidad

            CONTEXTUALIZACIÓN

        RESUMEN DE TODAS LAS TENDENCIAS ARA LLEGAR A ACOTAR LIME Y SHAP (model agnostic, local....)

            RESUMEN DE SHAP

            RESUMENT DE TODAS LAS COMBINACIONES DE LIME

                confirmación de que no hay combinación con fuzzy (aunque sí con shap pero tendrá alguna pega que motiva nuestro trabajo)

            PROPUESTA

        COMBINAR LIME CON FUZZY:

                COMBINACIÓN TIPO 1: ANFIS ES EL MODELO A EXPLICAR: LIME CON SU MODELO LINEAL ORIGINAL LO EXPLICA: ESTO VA A SER MALA IDEA

                    COMBINACIÓN TIPO 2: EL MODELO A EXPLICAR ES EL QUE SEA: FUZZY APORTA EL MODELO QUE EXPLICA

                    COMBINACIÓN TIPO 3:  EL MODELO A EXPLICAR ES EL QUE SEA: LIME CON SU MODELO LINEAL ORIGINAL Y SU RANKING DE VARIABLES PERO CON CON NÙMEROS
        DIFUSOS Y COMPARACION DIFUSA

          EXPERIMENTO

    EVALUACIÓN

    CONCLUSIONES

  Idealmente podría mostrarse algo así

                      SINERGIA     REDUNDANCIA    INUTILIDAD  ANTAGNISMO

    COMBINACIÓN TIPO 1:    X

    COMBINACIÓN TIPO 2:                X

    COMBINACIÓN TIPO 3:    X
```

TODOs

- pasarle un modelo Random Forest, NN, etc y conseguir explicaciones tanto locales como globales (sacando estas a partir de muchas explicaciones locales, mirar el pdf de lime otra vez)
- ver la diferencia entre explicabilidad e interpretabilidad (sobre todo puede ayudar para la memoria)
- Interpretable Machine learning pdf
- Estado del arte de XAI, fuzzy y porque fuzzy está en "su invierno"
- ANFIS o cualquier herramienta que genera reglas difusas desde datos (implementacion/libreria)

Librerías

In [1]:
```python
import pandas as pd
import numpy as np
import lime
from lime import lime_tabular, submodular_pick

from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.tree import export_text
import skfuzzy as fuzz
import matplotlib.pyplot as plt
from tqdm import tqdm
import seaborn as sns

import IPython
import sys

# Forzamos que IPython.core.display tenga el atributo display
IPython.core.display.display = IPython.display.display

import warnings
warnings.filterwarnings('ignore')
```

Prueba de generación de variables

In [2]:
```python
# def generar_dataset_polinomico(n_instancias=1000, n_variables=5, semilla=None):
#     """
#     Genera un dataset sintético con un número dinámico de variables y un target
#     basado en un polinomio generado aleatoriamente.
#     """
#     if semilla is not None:
#         np.random.seed(semilla)

#     # 1. Generar los nombres de las columnas (x1, x2, x3, ...)
#     columnas = [f'x{i}' for i in range(1, n_variables + 1)]

#     # 2. Generar la matriz de datos de entrada (features)
#     # Rango [-2, 2] para que las potencias altas no disparen los valores al infinito
#     X = np.random.uniform(-2, 2, size=(n_instancias, n_variables))

#     # 3. Generar aleatoriamente los coeficientes y las potencias para el polinomio
#     # Coeficientes entre -10 y 10
#     coeficientes = np.random.randint(-10, 11, size=n_variables)
#     # Potencias entre 1 y 4 (puedes ajustar este rango si quieres polinomios más complejos)
#     potencias = np.random.randint(1, 5, size=n_variables)

#     # 4. Calcular la variable de salida (target) 'y' y construir la fórmula en texto
#     y = np.zeros(n_instancias)
#     terminos_formula = []

#     for i in range(n_variables):
#         # Fórmula: c * (x^p)
#         y += coeficientes[i] * (X[:, i] ** potencias[i])

#         # Guardamos el término en texto para saber cuál es el "ground truth"
#         termino_str = f"{coeficientes[i]}*{columnas[i]}^{potencias[i]}"
#         terminos_formula.append(termino_str)

#     # Limpiamos un poco la fórmula generada para que se lea mejor matemáticamente
#     formula_final = "y = " + " + ".join(terminos_formula).replace("+ -", "- ")

#     # 5. Crear el DataFrame
#     df = pd.DataFrame(X, columns=columnas)
#     df['target'] = y

#     return df, formula_final, coeficientes, potencias
```

In [3]:
```python
def generar_dataset_polinomico(n_instancias=1000, semilla=None):
    """
    Genera un dataset sintético con un número dinámico de variables y un target
    basado en un polinomio generado aleatoriamente.
    """
    if semilla is not None:
        np.random.seed(semilla)

    # 1. Generar los nombres de las columnas (x1, x2, x3, ...)
    columnas = [f'x{i}' for i in range(1, 7)]

    # 2. Generar la matriz de datos de entrada (features)
    # Rango [1, 3] para que las potencias altas no disparen los valores al infinito
    X = np.random.uniform(1, 3, size=(n_instancias, 6))

    # 3. Generar aleatoriamente los coeficientes y las potencias para el polinomio
    # Coeficientes entre -3 y 3
    coeficientes = [-1, 2, -3, 3, 1, -2] # np.random.randint(-3, 4, size=6)
    # Potencias elegidas manualmente para conocer el polinomio exacto
    potencias = [2,1,5,5,2,1] # np.random.randint(1, 5, size=n_variables)

    # 4. Calcular la variable de salida (target) 'y' y construir la fórmula en texto
    y = np.zeros(n_instancias)
    terminos_formula = []

    for i in range(6):
        # Fórmula: c * (x^p)
        y += coeficientes[i] * (X[:, i] ** potencias[i])

        # Guardamos el término en texto para saber cuál es el "ground truth"
        termino_str = f"{coeficientes[i]}*{columnas[i]}^{potencias[i]}"
        terminos_formula.append(termino_str)

    # Limpiamos un poco la fórmula generada para que se lea mejor matemáticamente
    formula_final = "y = " + " + ".join(terminos_formula).replace("+ -", "- ")

    # 5. Crear el DataFrame
    df = pd.DataFrame(X, columns=columnas)
    df['target'] = y

    return df, formula_final, coeficientes, potencias
```

In [4]:
```python
# Vamos a generar un dataset con 6 variables
SEED = 1234
dataset, formula_real, coeficientes, potencias = generar_dataset_polinomico(n_instancias=5000, semilla=SEED) # n_variables=6,

# Imprimir la fórmula que el código se ha inventado para este dataset
print("La función real (Ground Truth) subyacente es:")
print(formula_real)
print("-" * 60)

# Mostrar el dataset resultante
display(dataset.head(20))
```

```
La función real (Ground Truth) subyacente es:
y = -1*x1^2 + 2*x2^1 - 3*x3^5 + 3*x4^5 + 1*x5^2 - 2*x6^1
------------------------------------------------------------
```

	x1	x2	x3	x4	x5	x6	target
0	1.383039	2.244218	1.875455	2.570717	2.559952	1.545185	273.247302
1	1.552929	2.603744	2.916279	2.751865	1.715635	2.001990	-157.633124
2	2.366926	2.425404	1.740502	2.122392	2.006166	1.027537	82.496865
3	2.545653	2.765282	1.729772	2.230792	1.150762	1.737648	116.176689
4	2.866280	2.302756	1.794405	2.577460	1.633672	2.136197	280.231612
5	2.738255	1.872347	2.604295	1.287534	2.408522	2.409163	-351.551121
6	1.437584	2.849735	1.884282	2.818632	1.119618	1.368574	464.608017
7	1.094711	2.349762	2.189250	2.066620	1.086648	2.122866	-37.341822
8	1.659337	2.005934	1.223789	2.214387	2.131889	1.013528	155.272583
9	2.234883	2.824246	2.581048	2.984163	2.917604	2.583928	370.319837
10	1.570502	2.249833	1.956188	1.391350	1.764635	1.107747	-67.361681
11	1.903297	2.964009	1.247885	1.238762	2.477046	2.174607	3.764979
12	1.943265	1.214254	1.458437	2.799930	1.833507	2.071703	494.322237
13	1.012417	1.601283	1.873786	2.224298	2.836396	2.251473	99.759207
14	2.411995	1.299667	2.492127	2.662014	2.267452	1.876620	110.811547
15	1.305146	2.136819	2.056449	2.902858	1.960718	2.005119	510.442185
16	2.073756	2.638404	1.114231	2.338843	2.534233	2.416231	207.369360
17	2.593734	2.115522	2.931673	1.294314	1.059294	2.187787	-644.531742
18	1.228131	2.901620	1.651415	1.387237	1.915623	2.840805	-19.151493
19	2.758138	1.505232	1.696018	1.365177	2.803592	2.413056	-29.436491

In [5]:
```python
max(dataset['target']), min(dataset['target'])
```

Out[5]:
```
(722.1297383027292, -725.1114338765286)
```

Division train test

In [6]:
```python
# 1. Preparar los datos
X = dataset.drop('target', axis=1)
y = dataset['target']

# Dividir en entrenamiento y test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Modelo predictor polinomico

In [7]:
```python
def mi_modelo_polinomico(X_array):
    X_np = np.array(X_array)

    # 2. Si es de 1 dimensión (como tu instancia_a_explicar),
    # lo forzamos a 2D añadiendo una dimensión extra (1 fila, N columnas)

    # lo comento ya que lo hago al pasarselo al modelo
    # if X_np.ndim == 1:
    #     X_np = X_np.reshape(1, -1)

    return np.sum(coeficientes * (X_np ** potencias), axis=1)
```

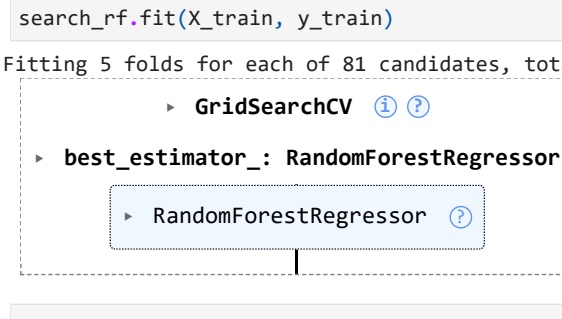
Random Forest

In [8]:
```python
#grid search para encontrar el mejor modelo de random forest
grid_rf = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}
```

In [9]:
```python
search_rf = GridSearchCV(
    estimator=RandomForestRegressor(random_state=SEED),
    param_grid=grid_rf,
    cv=5,
    n_jobs=-1,
    verbose=2
)
search_rf.fit(X_train, y_train)
```

```
Fitting 5 folds for each of 81 candidates, totalling 405 fits
```

Out[9]:



▸ GridSearchCV

▸ best_estimator_: RandomForestRegressor

▸ RandomForestRegressor

In [10]:
```python
rf = search_rf.best_estimator_
y_pred = rf.predict(X_test)
```

Explicador LIME

In [11]:
```python
# 3. Inicializar el explicador LIME
# OJO para tu TFM: LIME por defecto discretiza las variables continuas en cuartiles.
# Para comparar matemáticamente con un polinomio continuo, a veces es mejor desactivarlo.
explicador = lime.lime_tabular.LimeTabularExplainer(
    training_data=np.array(X_train),
    feature_names=X_train.columns,
    mode='regression',
    discretize_continuous=False # Fundamental para ver coeficientes lineales puros
)
```

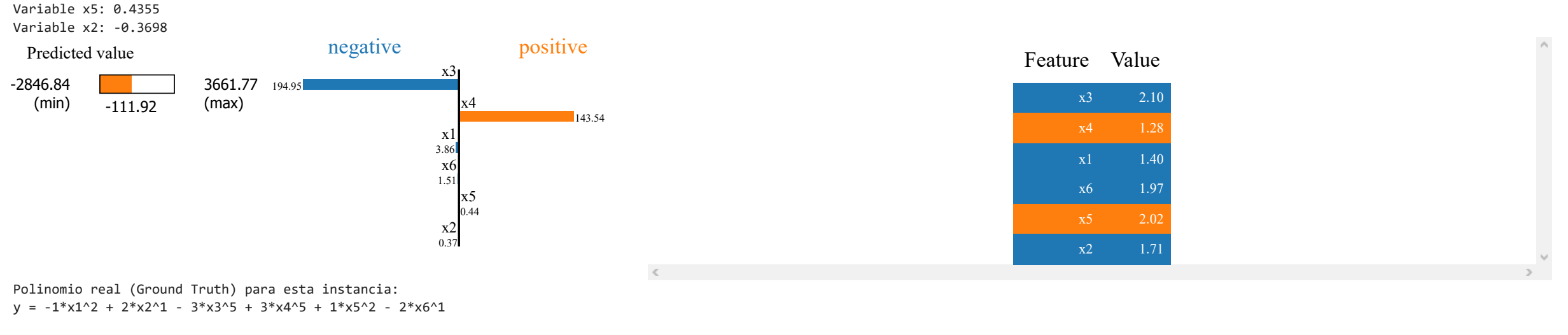
Ejemplo de explicación

In [12]:
```python
# 4. Elegir una instancia (fila) para explicar
indice_instancia = 0
instancia_a_explicar = X_test.iloc[indice_instancia]
valor_real = y_test.iloc[indice_instancia]
prediccion_modelo = y_pred[indice_instancia]

print(f"Explicando la instancia {indice_instancia} del conjunto de test:")
print(instancia_a_explicar.to_string())
print(f"\nValor Real (Polinomio): {valor_real:.4f}")
print(f"Predicción del Modelo: {prediccion_modelo:.4f}")
print("-" * 60)
```

```
Explicando la instancia 0 del conjunto de test:
x1    1.403933
x2    1.706193
x3    2.104602
x4    1.281884
x5    2.016106
x6    1.967475

Valor Real (Polinomio): -111.9164
Predicción del Modelo: -115.7882
------------------------------------------------------------
```

In [13]:
```python
# 5. Generar la explicación con LIME
explicacion = explicador.explain_instance(
    data_row=instancia_a_explicar,
    predict_fn=mi_modelo_polinomico,
    num_features=len(X_test.columns) # Que nos muestre el peso de todas las variables
)

# 6. Extraer y mostrar los pesos locales que LIME ha calculado
print("Pesos locales calculados por LIME (Aproximación lineal local):")
pesos_lime = explicacion.as_list()
for feature, weight in pesos_lime:
    print(f"Variable {feature}: {weight:.4f}")

# Visualizar la explicación en el notebook
explicacion.show_in_notebook(show_table=True)

print("\nPolinomio real (Ground Truth) para esta instancia:")
print(formula_real)
```

```
Pesos locales calculados por LIME (Aproximación lineal local):
Variable x3: -194.9458
Variable x4: 143.5378
Variable x1: -3.8554
Variable x6: -1.5058
Variable x5: 0.4355
Variable x2: -0.3698
```

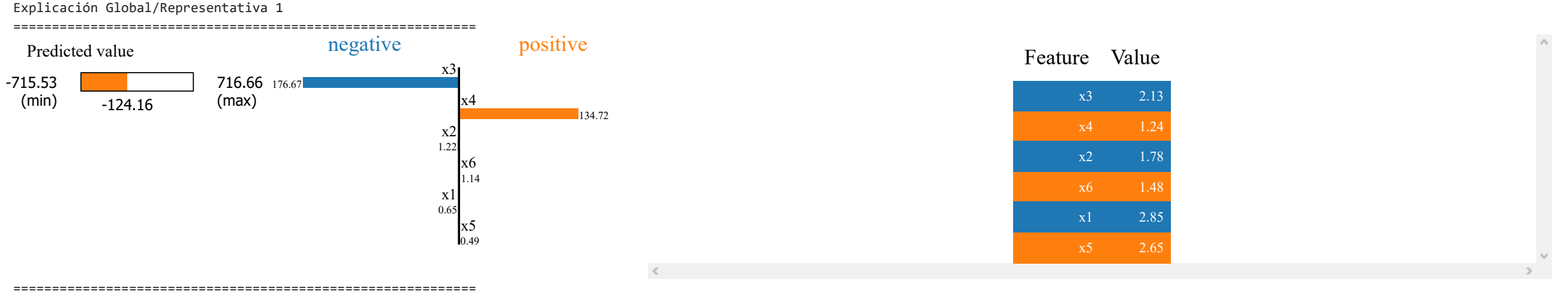


```
Polinomio real (Ground Truth) para esta instancia:
y = -1*x1^2 + 2*x2^1 - 3*x3^5 + 3*x4^5 + 1*x5^2 - 2*x6^1
```

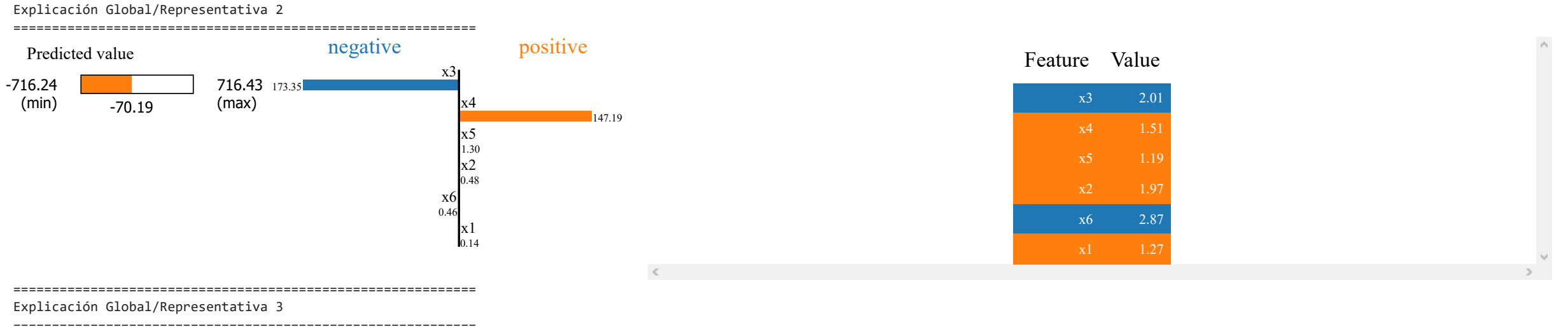
Explicación global con SP-LIME

In [14]:
```python
# Configuración
num_explicaciones_representativas = 5  # Cantidad de instancias a mostrar para entender el global
num_instancias_a_muestrear = len(X_test) #100 # Muestra aleatoria de X_test a evaluar (mayor = mejor, pero más lento)

# 1. Ejecutar el algoritmo Submodular Pick
sp_explainer = submodular_pick.SubmodularPick(
    explainer=explicador,
    data=X_test.values,         # Es recomendable pasar los valores como array de numpy
    predict_fn=rf.predict,      # Pasamos el modelo Random Forest
    sample_size=num_instancias_a_muestrear,
    num_features=len(X_test.columns),
    num_exps_desired=num_explicaciones_representativas
)

# 2. Visualizar las explicaciones globales (el conjunto representativo)
for i, exp in enumerate(sp_explainer.sp_explanations):
    print(f"\n{'='*60}")
    print(f"Explicación Global/Representativa {i+1}")
    print(f"{'='*60}")
    exp.show_in_notebook(show_table=True)
```

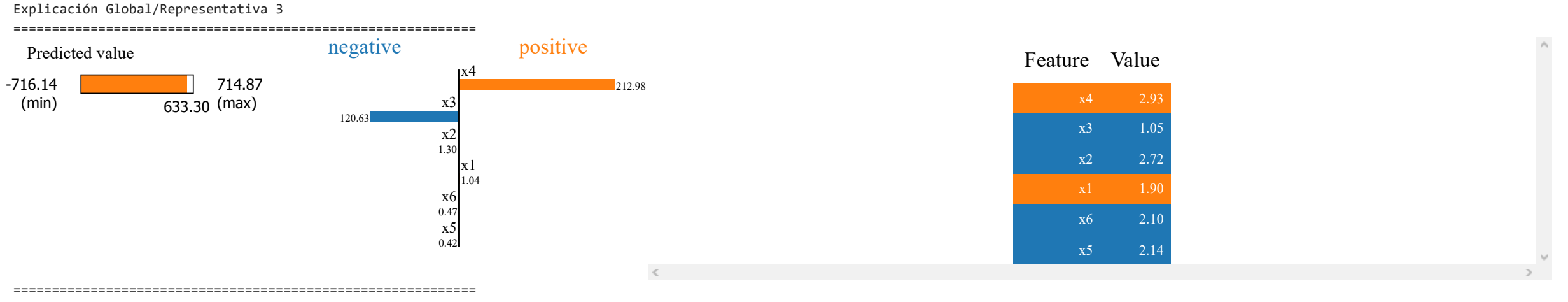
```
============================================================
Explicación Global/Representativa 1
============================================================
```



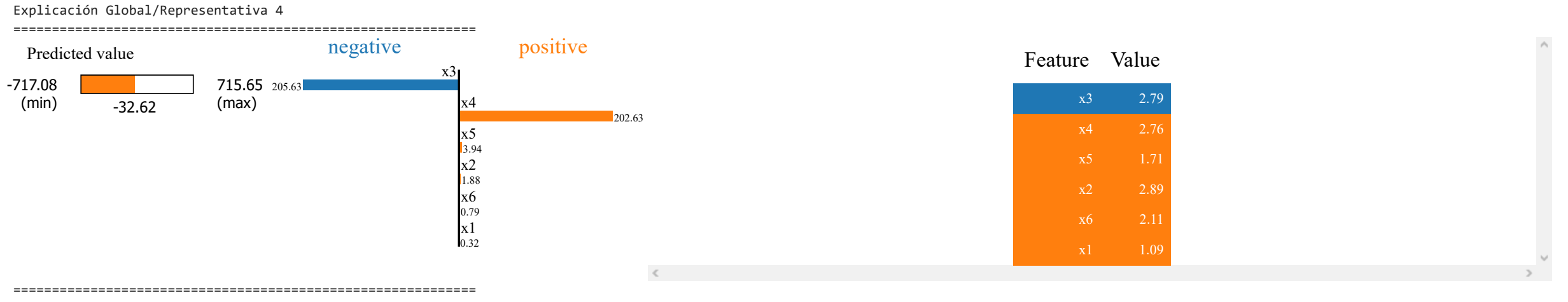
```
============================================================
Explicación Global/Representativa 2
============================================================
```



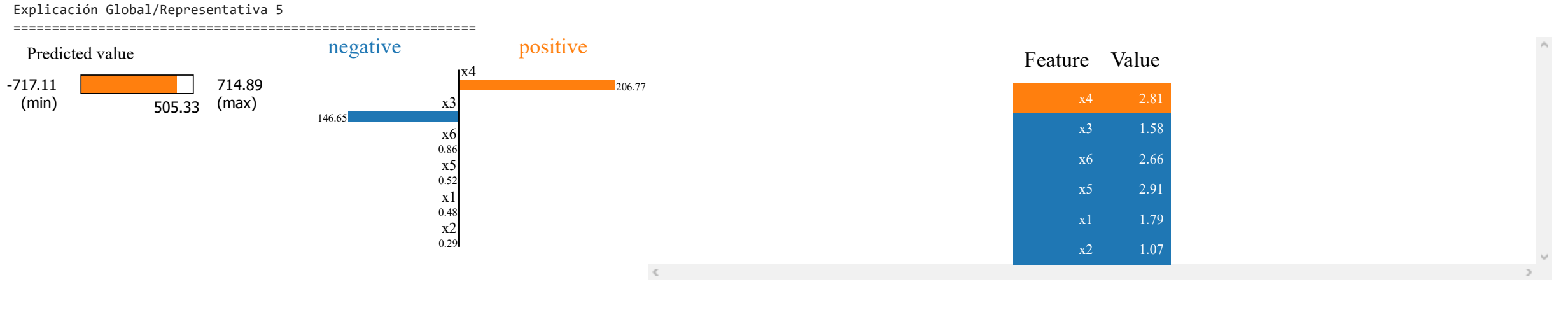
```
============================================================
Explicación Global/Representativa 3
============================================================
```



```
============================================================
Explicación Global/Representativa 4
============================================================
```



```
============================================================
Explicación Global/Representativa 5
============================================================
```



Obtencion de los vecinos más cercanos de LIME mediante un wrapper para una predicción local

In [15]:
```python
# 1. Variables globales para capturar los datos internos
datos_perturbados_lime = None
predicciones_rf_lime = None

# 2. Creamos la "función espía"
def funcion_espia(X):
    global datos_perturbados_lime, predicciones_rf_lime

    # LIME nos entrega aquí los miles de puntos sintéticos. ¡Los guardamos!
    datos_perturbados_lime = X

    # Ahora llamamos al Random Forest real para que LIME no se rompa
    preds = rf.predict(X)

    # Guardamos también las predicciones
    predicciones_rf_lime = preds
    return preds

# 3. Elegimos una instancia para la prueba (ej. la Instancia 0 de tu test)
instancia_prueba = X_train.iloc[0]

# 4. Ejecutamos LIME pero le pasamos nuestra función espía en lugar de rf.predict
print("Ejecutando LIME para generar el vecindario...")
explicacion = explicador.explain_instance(
    data_row=instancia_prueba,
    predict_fn=funcion_espia,      # ¡Clave! Usamos el wrapper
    num_features=len(X_train.columns)
)

# 5. ¡Magia! Convertimos los datos capturados en un DataFrame
df_vecindario = pd.DataFrame(datos_perturbados_lime, columns=X_train.columns)
df_vecindario['Prediccion_RF'] = predicciones_rf_lime

print(f"\n¡Éxito! LIME ha generado {len(df_vecindario)} instancias perturbadas locales.")
display(df_vecindario.head())
```

```
Ejecutando LIME para generar el vecindario...

¡Éxito! LIME ha generado 5000 instancias perturbadas locales.
```

	x1	x2	x3	x4	x5	x6	Prediccion_RF
0	2.370021	2.076388	2.062848	2.704845	1.750872	2.453485	320.179681
1	2.633000	1.970605	1.845458	2.176777	1.812620	2.150617	79.070786
2	2.355473	1.340441	2.457041	1.934774	1.686771	2.136332	-186.967636
3	1.873318	1.157183	1.746535	1.639769	1.659545	3.070703	-17.545552
4	1.901617	1.497615	2.263026	1.399306	2.499176	1.767872	-161.172097

Comprobamos con un arbol de decisión si podría tener sentido cambiar la regresion lineal de LIME por reglas difusas:

In [16]:
```python
# Entrenamos un árbol pequeño (max_depth=3 para que las reglas sean legibles)
# Esto simula lo que haría tu futuro motor de reglas difusas locales
extractor_reglas = DecisionTreeRegressor(max_depth=3, random_state=42)

# Entrenamos usando las variables originales contra lo que predijo el Random Forest
extractor_reglas.fit(df_vecindario[X_train.columns], df_vecindario['Prediccion_RF'])

# Extraemos y mostramos las reglas
print("=== REGLAS LOCALES EXTRAÍDAS DEL VECINDARIO ===")
reglas_texto = export_text(extractor_reglas, feature_names=list(X_train.columns))
print(reglas_texto)
```

```
=== REGLAS LOCALES EXTRAÍDAS DEL VECINDARIO ===
|--- x3 <= 2.52
|   |--- x4 <= 2.47
|   |   |--- x3 <= 2.03
|   |   |   |--- value: [46.97]
|   |   |--- x3 >  2.03
|   |   |   |--- value: [-96.80]
|   |--- x4 >  2.47
|   |   |--- x4 <= 2.76
|   |   |   |--- value: [277.73]
|   |   |--- x4 >  2.76
|   |   |   |--- value: [546.80]
|--- x3 >  2.52
|   |--- x4 <= 2.39
|   |   |--- x3 <= 2.79
|   |   |   |--- value: [-318.03]
|   |   |--- x3 >  2.79
|   |   |   |--- value: [-581.74]
|   |--- x4 >  2.39
|   |   |--- x4 <= 2.74
|   |   |   |--- value: [-194.62]
|   |   |--- x4 >  2.74
|   |   |   |--- value: [104.60]
```

Al imprimir estas reglas, verás algo parecido a:SI X_4 > 2.5 Y X_3 <= 1.2 ENTONCES Predicción = 540.2

Si compruebas que estas reglas rígidas capturan la no linealidad de tus potencias (grado 5) mucho mejor que los pesos lineales que te daba LIME, habrás validado técnicamente el Plan A. El siguiente reto será construir el motor difuso que suavice esas fronteras (X_4 > 2.5 se convertirá en X_4 es ALTO).

Una vez tengas estas reglas difusas locales funcionando para múltiples instancias, te enfrentarás al problema de cómo unirlas todas para crear el modelo global. Al analizar futuros grafos de selección para construir esa arquitectura global, considera que el algoritmo estándar de LIME (epsilon=0, puramente Greedy) tiene la tendencia a quedarse atascado en valores subóptimos debido a una falta de exploración.

Tener en tu poder el control absoluto de df_vecindario te da una ventaja inmensa: no tienes por qué depender de ese método de selección voraz. Puedes diseñar una estrategia de agregación difusa o un algoritmo genético que explore el espacio de soluciones de manera mucho más amplia, garantizando que el conjunto final de reglas difusas describa fielmente el comportamiento de tu Random Forest en todos los extremos de tu polinomio.

Ahora, pasemos a analizar la salida que has obtenido. ¡Es un resultado absolutamente brillante para tu tesis!

Fíjate en lo que acaba de hacer este simple árbol de decisión: ha ignorado por completo las variables X_1, X_2, X_5 y X_6. Al intentar explicar el vecindario local, el modelo basado en reglas ha deducido automáticamente que la predicción depende exclusiva y rotundamente de X_3 y X_4.

Si traducimos esas reglas matemáticas rígidas a lenguaje natural, el árbol ha descubierto exactamente el comportamiento de las potencias de grado 5 de tu polinomio:

La zona de máximo hundimiento (La penalización de X_3):

- Regla: |--- x3 > 2.52 y |--- x4 <= 2.39 y |--- x3 > 2.79 → value: [-581.74]
- Traducción: Si X_3 toma valores muy altos (cercanos a 3) y X_4 se mantiene bajo, la predicción se desploma a -581. El término $-3x_3^5$ domina por completo.

La zona de máxima explosión (El impulso de X_4):

- Regla: |--- x3 <= 2.52 y |--- x4 > 2.47 y |--- x4 > 2.76 → value: [546.80]
- Traducción: Si es X_4 quien toma valores muy altos y X_3 se mantiene bajo, la predicción se dispara a +546. El término $+3x_4^5$ toma el control.

La zona de conflicto (Interacciones):

- Regla: |--- x3 > 2.52 y |--- x4 > 2.39 y |--- x4 > 2.74 → value: [104.60]
- Traducción: ¿Qué pasa si ambas variables tienen valores altos simultáneamente? Al ser de signos opuestos, compiten entre sí. El árbol refleja que el resultado se modera muchísimo (104.60) porque se contrarrestan.

Esta prueba empírica demuestra que las reglas superan al modelo lineal clásico de LIME para representar interacciones complejas. Un modelo lineal habría dado un peso estático a X_3 y a X_4, pero nunca te habría explicado cómo interactúan entre ellas cuando ambas crecen.

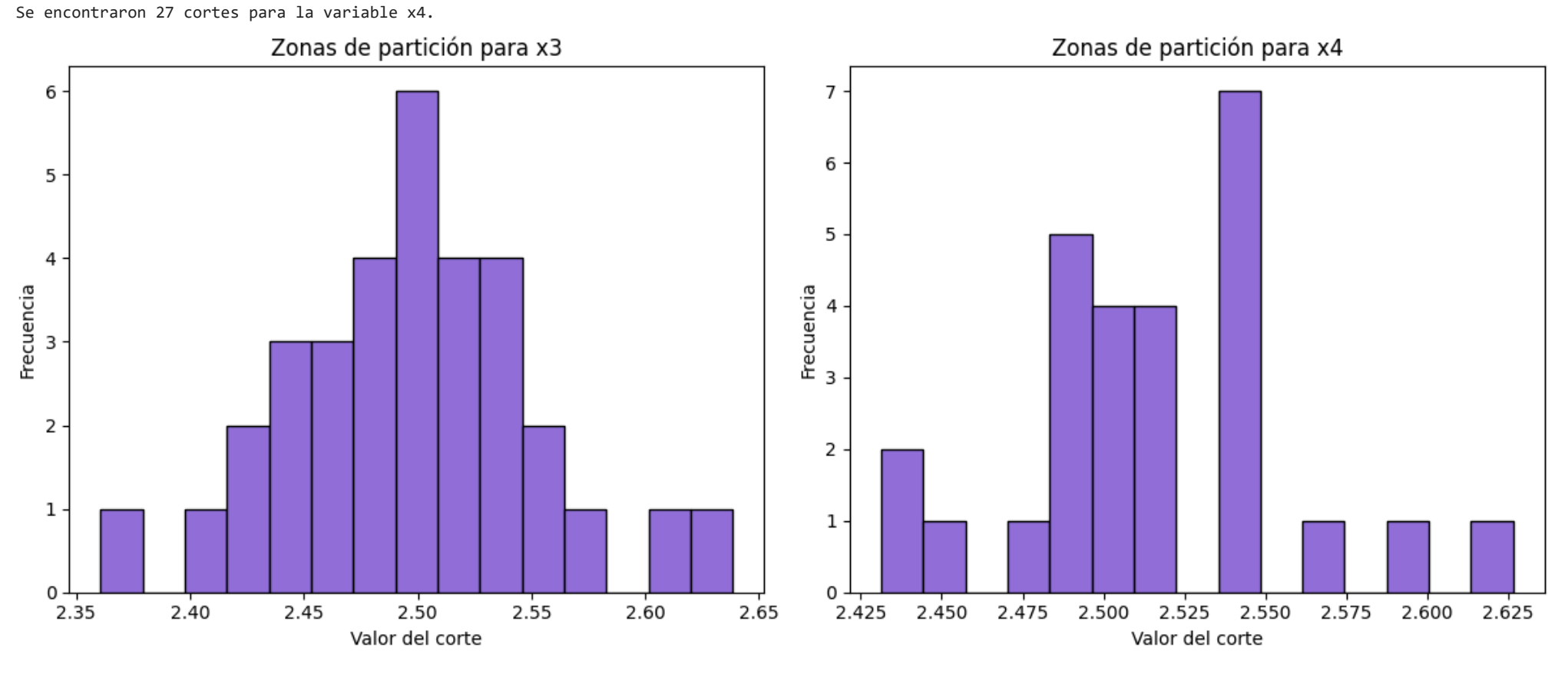
Bucle de obtención de reglas para N instancias diferentes

Realizamos este bucle de modo que se puedan ver qué patrones extrae el sistema en diferentes zonas del polinomio.

El objetivo de este primer bloque de código es automatizar lo que hiciste a mano en el paso anterior, pero para múltiples instancias simultáneamente, y recolectar los "puntos de corte" (los valores numéricos donde el árbol decide separar las ramas) para las variables críticas.Al considerar que el algoritmo de selección interno que propone LIME para representaciones globales (el método voraz con $\epsilon = 0$) tiende a estancarse en valores subóptimos por falta de exploración, en esta recolección masiva vamos a iterar directamente sobre una muestra diversa de tu conjunto de test. Esto nos garantiza capturar la variabilidad real del polinomio en todo su dominio, evitando que los puntos de corte se sesguen hacia una sola región.Añade la siguiente celda a tu cuaderno. Este código utiliza la misma "función espía" que creamos antes, pero ahora se sumerge en la estructura interna del árbol de decisión de Scikit-Learn para extraer matemáticamente los umbrales numéricos de las reglas.

In [17]:
```python
# 1. Variables globales para nuestra función espía (las volvemos a definir para que estén limpias)
datos_perturbados_lime = None
predicciones_rf_lime = None

# 2. Almacenes para los puntos de corte de nuestras variables estrella
dict_cortes = {}

num_instancias_a_evaluar = 20 # Cambiar a len(X_train) para evaluar todas las instancias del test (se recomienda usar un subconjunto para pruebas rápidas)
print(f"Extrayendo reglas locales de {num_instancias_a_evaluar} instancias...\n")

for i in range(num_instancias_a_evaluar):
    instancia = X_train.iloc[i]

    # Generamos el vecindario silenciosamente (sin imprimir nada)
    explicacion = explicador.explain_instance(
        data_row=instancia,
        predict_fn=funcion_espia,
        num_features=len(X_train.columns)
    )

    # Reconstruimos los datos
    df_vecindario = pd.DataFrame(datos_perturbados_lime, columns=X_train.columns)

    # Entrenamos el árbol local (max_depth=2 para buscar los cortes más drásticos)
    dt = DecisionTreeRegressor(max_depth=2, random_state=SEED)
    dt.fit(df_vecindario, predicciones_rf_lime)

    # Extraemos los puntos de corte directamente de la estructura del árbol
    for node_id in range(dt.tree_.node_count):
        # Si el nodo tiene hijos, es un nodo de decisión (un "corte")
        if dt.tree_.children_left[node_id] != dt.tree_.children_right[node_id]:
            feature_index = dt.tree_.feature[node_id]
            feature_name = X_train.columns[feature_index]
            threshold = dt.tree_.threshold[node_id]

            if feature_name not in dict_cortes:
                dict_cortes[feature_name] = []
            dict_cortes[feature_name].append(threshold)

# --- IMPRESIÓN DINÁMICA ---
print("--- Resumen de extracción ---")
# Ordenamos el diccionario por la cantidad de cortes (las más importantes primero)
dict_cortes = dict(sorted(dict_cortes.items(), key=lambda item: len(item[1]), reverse=True))

for feature, cortes in dict_cortes.items():
    print(f"Se encontraron {len(cortes)} cortes para la variable {feature}.")

# --- VISUALIZACIÓN DINÁMICA ---
num_features = len(dict_cortes)

if num_features > 0:
    # Creamos dinámicamente tantas columnas de gráficos como variables encontradas
    fig, axes = plt.subplots(1, num_features, figsize=(6 * num_features, 5))

    # Si solo hay 1 variable, axes no es un array, así que lo forzamos a ser lista
    if num_features == 1:
        axes = [axes]

    for ax, (feature, cortes) in zip(axes, dict_cortes.items()):
        ax.hist(cortes, bins=15, color='mediumpurple', edgecolor='black')
        ax.set_title(f'Zonas de partición para {feature}')
        ax.set_xlabel('Valor del corte')
        ax.set_ylabel('Frecuencia')

    plt.tight_layout()
    plt.show()
else:
    print("Los árboles no encontraron ningún corte válido (modelo demasiado simple).")
```

```
Extrayendo reglas locales de 20 instancias...

--- Resumen de extracción ---
Se encontraron 33 cortes para la variable x3.
Se encontraron 27 cortes para la variable x4.
```



ANOTACIÓN IMPORTANTE

Usamos lime solamente para generar las reglas pero no hacemos caso de sus predicciones. Entonces el proceso de "generación de reglas" y su aplicación se podrían sustituir en un futuro en lugar del regresor lineal de lime

Explicación de Gemini:

Si en un futuro decidieras publicar tu propia librería en Python (por ejemplo, haciendo un fork del repositorio oficial de LIME en GitHub), literalmente irías a su código fuente, borrarías las líneas donde entrenan el modelo lineal, e insertarías la llamada a tu generador de reglas difusas. De cara al usuario final, la herramienta funcionaría igual: le pasas una instancia y te devuelve una explicación, pero en lugar de un gráfico de barras confuso con pendientes locales, le devolvería un conjunto de reglas lingüísticas claras.

Aquí tienes la hoja de ruta corregida y ordenada, con cada paso resumido en una sola frase tal y como pedías:

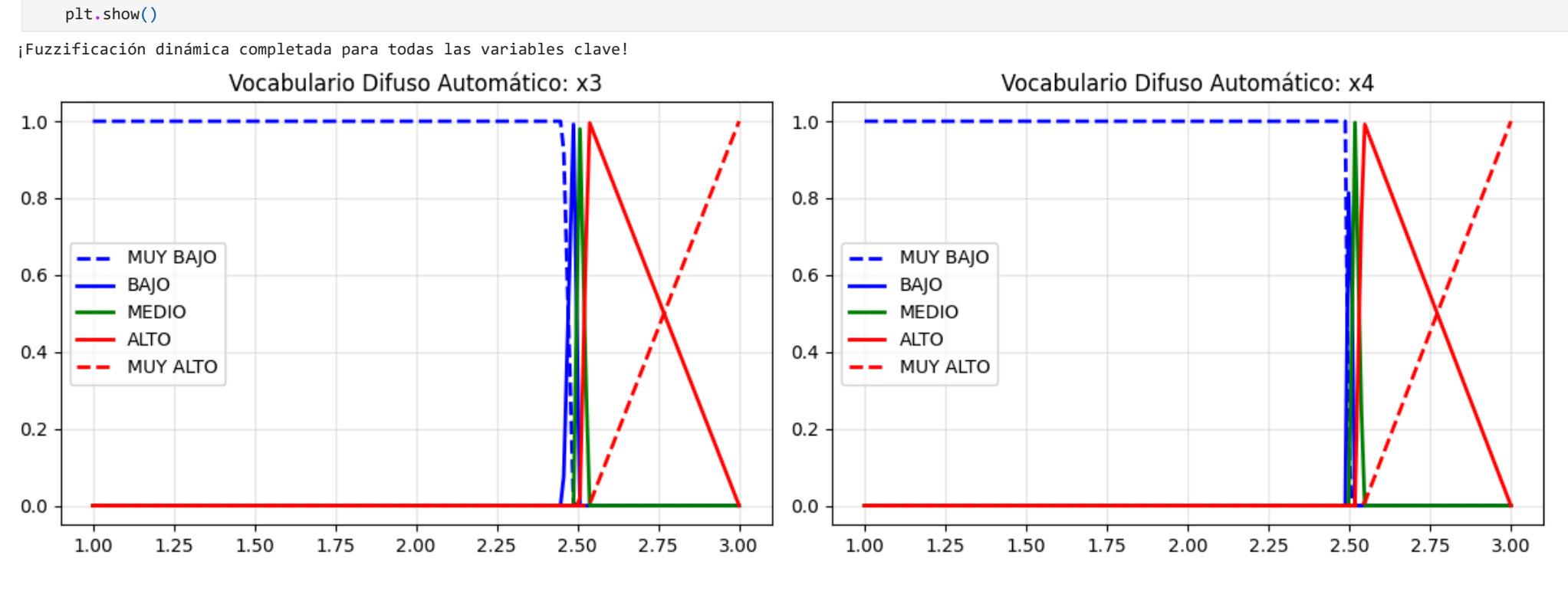
- Fuzzificación del dominio: Definir matemáticamente las funciones de pertenencia (MUY ALTO, ALTO, MEDIO, BAJO, MUY BAJO) a partir de los histogramas para establecer el vocabulario difuso del sistema.
- Extracción y traducción masiva: Extraer las reglas rígidas locales de las perturbaciones generadas por LIME para todo el conjunto de test y traducirlas automáticamente al vocabulario difuso.
- Agregación global: Agrupar estas miles de reglas locales en un conjunto representativo mediante técnicas avanzadas, evitando que el clásico algoritmo de optimización puramente voraz con epsilon=0 se quede atascado en valores subóptimos por su falta de exploración.
- Construcción del modelo: Diseñar el Sistema de Inferencia Difuso (FIS) alimentado por esas reglas globales para que sea capaz de predecir valores y emitir explicaciones lingüísticas.
- Encapsulamiento de la herramienta: Modificar el código para sustituir completamente la regresión lineal de LIME por nuestro nuevo método de inferencia difuso, logrando un explicador totalmente agnóstico.

Fuzzificación de variables encontradas en las reglas

Revisar a partir de aquí

In [18]:
```python
# Diccionario maestro que guardará todo nuestro vocabulario difuso
dict_vocabulario_difuso = {}

# Iteramos sobre el diccionario dinámico de cortes que ya teníamos
for feature, cortes in dict_cortes.items():

    # 1. Definimos el universo de forma dinámica buscando el min y max real en el dataset
    min_val = X_train[feature].min()
    max_val = X_train[feature].max()
    universo = np.linspace(min_val, max_val, 200) # 200 puntos de resolución


    p20, p40, p60, p80 = np.percentile(cortes, [20, 40, 60, 80])

    dict_vocabulario_difuso[feature] = {
        'universo': universo,
        'MUY BAJO': fuzz.trapmf(universo, [min_val, min_val, p20, p40]),
        'BAJO': fuzz.trimf(universo, [p20, p40, p60]),
        'MEDIO': fuzz.trimf(universo, [p40, p60, p80]),
        'ALTO': fuzz.trimf(universo, [p60, p80, max_val]),
        'MUY ALTO': fuzz.trapmf(universo, [p80, max_val, max_val, max_val])
    }

print("¡Fuzzificación dinámica completada para todas las variables clave!")

# --- VISUALIZACIÓN DINÁMICA ---
num_features = len(dict_vocabulario_difuso)

if num_features > 0:
    fig, axes = plt.subplots(1, num_features, figsize=(6 * num_features, 4))
    if num_features == 1:
        axes = [axes]

    for ax, (feature, fuzzy_sets) in zip(axes, dict_vocabulario_difuso.items()):
        universo = fuzzy_sets['universo']
        ax.plot(universo, fuzzy_sets['MUY BAJO'], 'b--', linewidth=2, label='MUY BAJO')
        ax.plot(universo, fuzzy_sets['BAJO'], 'b', linewidth=2, label='BAJO')
        ax.plot(universo, fuzzy_sets['MEDIO'], 'g', linewidth=2, label='MEDIO')
        ax.plot(universo, fuzzy_sets['ALTO'], 'r', linewidth=2, label='ALTO')
        ax.plot(universo, fuzzy_sets['MUY ALTO'], 'r--', linewidth=2, label='MUY ALTO')

        ax.set_title(f'Vocabulario Difuso Automático: {feature}')
        ax.legend()
        ax.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()
```

```
¡Fuzzificación dinámica completada para todas las variables clave!
```



Supuesto pipeline a partir de ahora (obtencion de reglas rigidas, fuzzificacion de las variables, fuzzificacion de las reglas, etc.)

Probablemente haya (hay) cosas repetidas asique revisar bien.

In [19]:
```python
# ==========================================
# CONFIGURACIÓN Y VARIABLES GLOBALES
# ==========================================
datos_perturbados_lime = None
predicciones_rf_lime = None

def funcion_espia(X):
    global datos_perturbados_lime, predicciones_rf_lime
    datos_perturbados_lime = X
    preds = rf.predict(X)
    predicciones_rf_lime = preds
    return preds

# Estructuras de almacenamiento global
dict_cortes = {}             # Para calcular los percentiles
lista_reglas_crudas = []     # El "saco" de reglas rígidas

# Función auxiliar para extraer caminos de decisión estructurados del árbol
def extraer_reglas_arbol(dt, nombres_variables, id_instancia):
    tree_ = dt.tree_

    def recorrer(nodo, camino_actual):
        # Si es un nodo hoja, guardamos la regla completa
        if tree_.children_left[nodo] == tree_.children_right[nodo]:
            prediccion = tree_.value[nodo][0][0]
            lista_reglas_crudas.append({
                'instancia_origen': id_instancia,
                'condiciones': camino_actual,
                'prediccion': prediccion
            })
            return

        # Si es un nodo de decisión, extraemos el corte
        feature_name = nombres_variables[tree_.feature[nodo]]
        threshold = tree_.threshold[nodo]

        # Alimentamos el diccionario dinámico de cortes
        if feature_name not in dict_cortes:
            dict_cortes[feature_name] = []
        dict_cortes[feature_name].append(threshold)

        # Explorar rama izquierda (<=) y derecha (>)
        recorrer(tree_.children_left[nodo], camino_actual + [(feature_name, '<=', threshold)])
        recorrer(tree_.children_right[nodo], camino_actual + [(feature_name, '>', threshold)])

    recorrer(0, []) # Iniciar desde la raíz

# ==========================================
# FASE 1: EXTRACCIÓN MASIVA
# ==========================================
num_instancias_a_procesar = 50 # Cambiar a len(X_train) para ejecución final
print(f"--- FASE 1: Extrayendo vecindarios y reglas de {num_instancias_a_procesar} instancias ---")

for i in range(num_instancias_a_procesar):
    instancia = X_train.iloc[i]

    # 1. Generar vecindario con LIME (usando la función espía)
    _ = explicador.explain_instance(
        data_row=instancia,
        predict_fn=funcion_espia,
        num_features=len(X_train.columns)
    )

    # 2. Reconstruir datos locales
    df_vecindario = pd.DataFrame(datos_perturbados_lime, columns=X_train.columns)

    # 3. Entrenar el traductor local (Árbol rígido)
    dt = DecisionTreeRegressor(max_depth=2, random_state=42)
    dt.fit(df_vecindario, predicciones_rf_lime)

    # 4. Extraer cortes y reglas al mismo tiempo
    extraer_reglas_arbol(dt, X_train.columns, id_instancia=i)

print(f"Extracción completada. Se generaron {len(lista_reglas_crudas)} reglas rígidas locales.\n")


# ==========================================
# FASE 2: FUZZIFICACIÓN DINÁMICA (VOCABULARIO)
# ==========================================
print("--- FASE 2: Construyendo Vocabulario Difuso Universal ---")
dict_vocabulario_difuso = {}

for feature, cortes in dict_cortes.items():
    min_val = X_train[feature].min()
    max_val = X_train[feature].max()
    universo = np.linspace(min_val, max_val, 200)

    p20, p40, p60, p80 = np.percentile(cortes, [20, 40, 60, 80])

    dict_vocabulario_difuso[feature] = {
        'universo': universo,
        'MUY BAJO': fuzz.trapmf(universo, [min_val, min_val, p20, p40]),
        'BAJO': fuzz.trimf(universo, [p20, p40, p60]),
        'MEDIO': fuzz.trimf(universo, [p40, p60, p80]),
        'ALTO': fuzz.trimf(universo, [p60, p80, max_val]),
        'MUY ALTO': fuzz.trapmf(universo, [p80, max_val, max_val, max_val])
    }

print("Vocabulario construido basándose en la estadística global de los cortes.\n")


# ==========================================
# FASE 3: TRADUCTOR LINGÜÍSTICO EN MEMORIA
# ==========================================
print("--- FASE 3: Traducción de Reglas Rígidas a Lógica Difusa ---")
lista_reglas_difusas = []

for regla in lista_reglas_crudas:
    condiciones_difusas = []

    for (feature, operador, umbral) in regla['condiciones']:
        vocabulario = dict_vocabulario_difuso[feature]
        universo = vocabulario['universo']

        # Encontramos el índice más cercano al umbral en nuestro universo
        idx_umbral = (np.abs(universo - umbral)).argmin()

        # Evaluamos el grado de pertenencia de ese umbral a cada conjunto
        # pertenencia = {
        #     'MUY BAJO': vocabulario['MUY BAJO'][idx_umbral],
        #     'BAJO': vocabulario['BAJO'][idx_umbral],
        #     'MEDIO': vocabulario['MEDIO'][idx_umbral],
        #     'ALTO': vocabulario['ALTO'][idx_umbral],
        #     'MUY ALTO': vocabulario['MUY ALTO'][idx_umbral]
        # }

        # Lo hacemos dinámico para no tener que escribirlo a mano cada vez
        pertenencia = {}
        for etiqueta in vocabulario.keys():
            if etiqueta != 'universo': # Ignoramos el array del universo
                pertenencia[etiqueta] = vocabulario[etiqueta][idx_umbral]

        # Elegimos la etiqueta lingüística ganadora (la de mayor pertenencia)
        etiqueta_ganadora = max(pertenencia, key=pertenencia.get)
        condiciones_difusas.append(f"{feature} es {etiqueta_ganadora}")

    # Unimos las condiciones con un AND lógico
    regla_texto = " Y ".join(condiciones_difusas)
    pred_redondeada = round(regla['prediccion'], 2)

    lista_reglas_difusas.append(f"SI {regla_texto} ENTONCES Predicción = {pred_redondeada}")

# Mostramos una muestra del banco de reglas final
print("¡Traducción completada! Muestra aleatoria del banco global de reglas difusas:")
for i in range(5):
    import random
    print(random.choice(lista_reglas_difusas))
```

```
--- FASE 1: Extrayendo vecindarios y reglas de 50 instancias ---
Extracción completada. Se generaron 200 reglas rígidas locales.

--- FASE 2: Construyendo Vocabulario Difuso Universal ---
Vocabulario construido basándose en la estadística global de los cortes.

--- FASE 3: Traducción de Reglas Rígidas a Lógica Difusa ---
¡Traducción completada! Muestra aleatoria del banco global de reglas difusas:
SI x4 es MEDIO Y x3 es MUY BAJO ENTONCES Predicción = 436.47
SI x4 es MEDIO Y x3 es BAJO ENTONCES Predicción = 418.28
SI x4 es MUY BAJO Y x3 es BAJO ENTONCES Predicción = -429.01
SI x3 es ALTO Y x4 es ALTO ENTONCES Predicción = 1.33
SI x3 es ALTO Y x4 es BAJO ENTONCES Predicción = 389.28
```